Institute of Technology of Cambodia

Department of Applied Mathematic and Statistic

Data Engineering Pipeline for Credit Risk Modelling

Architecture, Technology Stack, Pipeline Design, and RAG Chat Interface of the Lending Club Credit Risk System

Group Members

Name	Student ID	Role
SOPHON Rachana	e20220725	Group Member
LOT Soklang	e20221464	Group Member
DUK Pagnarith	e20220184	Group Member
LENG Devid	e20220888	Group Member

Under the supervision of

Lecturer Vesal Khean

Academic Year: 2025–2026

Abstract

This paper describes the design, implementation, and rationale of the data engineering pipeline underpinning a production-grade credit risk modelling system built on the Lending Club peer-to-peer lending dataset. The system ingests 2,260,668 loan records (1.6 GB raw CSV) and transforms them into regulatory-grade model inputs through six Apache Airflow orchestration DAGs, two Apache Spark jobs, a five-schema PostgreSQL data warehouse, and a MinIO S3-compatible data lake.

The six pipelines are: (1) **Ingestion** — column-projected PySpark ingest from MinIO to PostgreSQL in approximately 7 seconds; (2) **Cleaning** — removal of 57 leaking, null, and irrelevant columns and creation of three target variables with an out-of-time split flag; (3) **Feature Engineering** — Weight of Evidence (WoE) encoding on the training set, producing 56 binary dummies from 16 raw predictors; (4) **Model Training** — PD logistic regression scorecard with iterative p-value selection and intercept calibration, LGD two-stage regression, and EAD linear regression, all tracked in MLflow; (5) **Batch Scoring** — applying all models to the full out-of-time portfolio and computing Expected Loss, IFRS 9 ECL, and Basel III AIRB capital per loan; and (6) **Monitoring** — daily Population Stability Index (PSI) and Characteristic Stability Index (CSI) computation with Grafana alerting.

Each pipeline layer is described in terms of its business purpose, the technology chosen to implement it, and the specific data transformations it performs. The paper also describes the five-layer database schema (raw, staging, features, models, risk), the MLflow model registry governance workflow, and the real-time FastAPI scoring service. All six pipelines were verified end-to-end on the real dataset with zero errors (v4 run, 2026-06-07), producing a calibrated PD scorecard (AUC = 0.694, Gini = 0.387 on OOT), an LGD two-stage model (mean LGD = 92.19%), and total expected loss of \$0.808B on a \$3.26B portfolio.

This updated version introduces Chapter 9: a Retrieval-Augmented Generation (RAG) chat interface that allows users to query both research reports and scored loan data interactively via a Streamlit dashboard, powered by ChromaDB vector stores and the DeepSeek large language model.

Keywords: data engineering, Apache Airflow, Apache Spark, PySpark, MLflow, PostgreSQL, MinIO, FastAPI, credit risk, ETL pipeline, model monitoring, PSI, RAG, ChromaDB, LangChain, Streamlit, DeepSeek.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbrev.	Definition	Abbrev.	Definition
AIRB	Advanced Internal Ratings-Based (Basel III)	JDBC	Java Database Connectivity
AUC	Area Under the ROC Curve	KS	Kolmogorov-Smirnov statistic
CCF	Credit Conversion Factor	LGD	Loss Given Default
CSI	Characteristic Stability Index	LLM	Large Language Model
DAG	Directed Acyclic Graph	MinIO	Minimal I/O (S3-compatible object store)
DR	Default Rate	MLflow	ML lifecycle tracking platform
EAD	Exposure at Default	OOT	Out-of-Time test set
EBA	European Banking Authority	PD	Probability of Default
ECOA	Equal Credit Opportunity Act	PDO	Points to Double the Odds
ECL	Expected Credit Loss (IFRS 9)	PiT	Point-in-Time PD
EL	Expected Loss	PSI	Population Stability Index
ETL	Extract, Transform, Load	RAG	Retrieval-Augmented Generation
FICO	Fair Isaac Corporation credit score	REST	Representational State Transfer
Gini	$2 \times \text{AUC} - 1$	RWA	Risk-Weighted Assets
HL	Hosmer-Lemeshow statistic	SICR	Significant Increase in Credit Risk
IFRS	Intl. Financial Reporting Standards	SR	Supervisory Regulation (Fed. Reserve)
IRB	Internal Ratings-Based	TtC	Through-the-Cycle PD
IV	Information Value	WoE	Weight of Evidence

Chapter 1

Introduction

1.1 Problem Statement

Credit risk modelling literature has made substantial progress in improving discrimination metrics — Gini coefficients, KS statistics, and AUC values for logistic regression scorecards are well-documented across dozens of published papers. However, the engineering infrastructure required to *deploy and maintain* a credit risk model in production is almost entirely absent from academic work. A model is not a product. A trained logistic regression that achieves Gini = 0.387 on a holdout set does not, by itself, produce IFRS 9 provisions, compute Basel III capital requirements, monitor its own population stability, or serve real-time loan decisions at sub-200 ms latency.

This paper addresses that gap by documenting a complete, end-to-end data engineering system for credit risk modelling, built on the Lending Club peer-to-peer lending dataset. The central engineering challenge is the transformation of a static, 2.26-million-row CSV file into a governed, monitored, auditable pipeline that produces regulatory-grade outputs — outputs that a real bank could submit to a prudential regulator. Solving this challenge requires decisions at every layer: storage technology, orchestration framework, schema design, feature encoding discipline, model governance, and real-time serving architecture.

1.2 Research Objectives

This paper pursues the following five objectives:

1. **Design a production-grade data pipeline architecture** that transforms raw loan records into model-ready features through clearly separated, auditable pipeline stages orchestrated by Apache Airflow.

2. **Implement a five-schema data warehouse** reflecting a medallion architecture (Bronze → Silver → Gold) that preserves data lineage and supports independent re-runs of any pipeline stage.

3. **Train and calibrate regulatory-grade credit risk models** — a PD logistic

regression scorecard with p-value-based feature selection and intercept calibration, a two-stage LGD model, and an EAD regression — all tracked through MLflow.

4. **Compute IFRS 9 Expected Credit Loss and Basel III AIRB capital** at the per-loan level, producing the full regulatory output stack that a mid-size financial institution would require for prudential reporting.
5. **Deploy a Retrieval-Augmented Generation (RAG) chat interface** that allows any team member to query the pipeline's documentation and scored loan data in natural language, reducing the time to answer ad-hoc analytical questions from minutes to seconds.

1.3 Motivation

The Lending Club dataset covers 2,260,668 loan applications from 2007 to 2018, spanning 151 columns and approximately 1.6 gigabytes in raw CSV form [5]. Every stage of the pipeline — from raw data ingestion through to daily model monitoring — is documented: the technology chosen, the rationale for that choice, the specific transformations performed, and the artefacts produced.

The system is built to be reproducible on any machine with Docker installed, using Docker Compose to orchestrate all 11 services. This makes it suitable for academic demonstration, regulatory review, and as a reference architecture for practitioners building their first production credit risk pipeline.

1.4 Data Engineering in Credit Risk

In the context of credit risk modelling, data engineering refers to the set of processes, tools, and infrastructure that:

1. **Move data** from its raw source (a CSV file, a database, a real-time loan origination system) into a form usable by statistical models.
2. **Transform data** by cleaning, validating, encoding, and splitting it in a way that prevents target leakage, preserves regulatory audit trails, and maintains reproducibility.
3. **Orchestrate** the sequence of transformations as a directed acyclic graph (DAG) of tasks, so that each step depends on the successful completion of the previous step and can be retried, monitored, and re-run independently.
4. **Serve** model outputs in real time to operational systems (loan origination platforms, IFRS 9 provisioning engines, Basel III capital reporting) with sub-second latency.

5. **Monitor** both the data distribution and the model outputs over time, detecting population shift and model degradation before they produce incorrect credit decisions.

These five responsibilities map directly to the six Airflow DAGs described in this paper. The sixth DAG (feature engineering) sits between cleaning and training and is separately described because it introduces the most domain-specific logic: Weight of Evidence encoding fitted exclusively on the training set.

1.5 Paper Structure

Chapter 2 describes the overall system architecture and the technology stack. Chapter 3 describes the five-schema PostgreSQL data warehouse design. Chapter 4 describes each of the six Airflow DAGs in detail. Chapter 5 describes the PySpark transformation jobs. Chapter 6 describes the MLflow experiment tracking and model registry. Chapter 7 describes the real-time FastAPI scoring service. Chapter 8 describes the monitoring infrastructure. Chapter 9 introduces the RAG Chat Interface. Chapter 10 discusses design decisions and limitations. Chapter 11 concludes.

Chapter 2

System Architecture

2.1 Architecture Overview

The system is organised into five horizontal layers, each corresponding to a distinct engineering concern:

1. **Ingestion and Storage Layer** — Raw CSV files are stored in MinIO (an on-premise, S3-compatible object store) and ingested into a five-schema PostgreSQL data warehouse via PySpark.
2. **Processing Layer** — Apache Spark (PySpark) performs large-scale data transformation: cleaning, feature engineering, and batch scoring. PySpark is chosen because the 2.26-million-row dataset with 151 columns exceeds comfortable single-machine Pandas performance at the feature engineering stage.
3. **Orchestration Layer** — Apache Airflow schedules, sequences, and monitors the six processing DAGs, providing full audit trails of every pipeline run, task-level retries, and SLA tracking.
4. **ML and Serving Layer** — MLflow tracks all model training experiments and manages the model registry. A FastAPI service provides real-time credit scoring via a REST endpoint, with Redis caching hot applicant features.
5. **Monitoring Layer** — Prometheus collects time-series metrics (PSI, CSI, API latency, pipeline run status). Grafana visualises these metrics and fires alert notifications when thresholds are breached.

All five layers run as Docker containers orchestrated by Docker Compose, making the full stack reproducible on any machine with Docker installed.

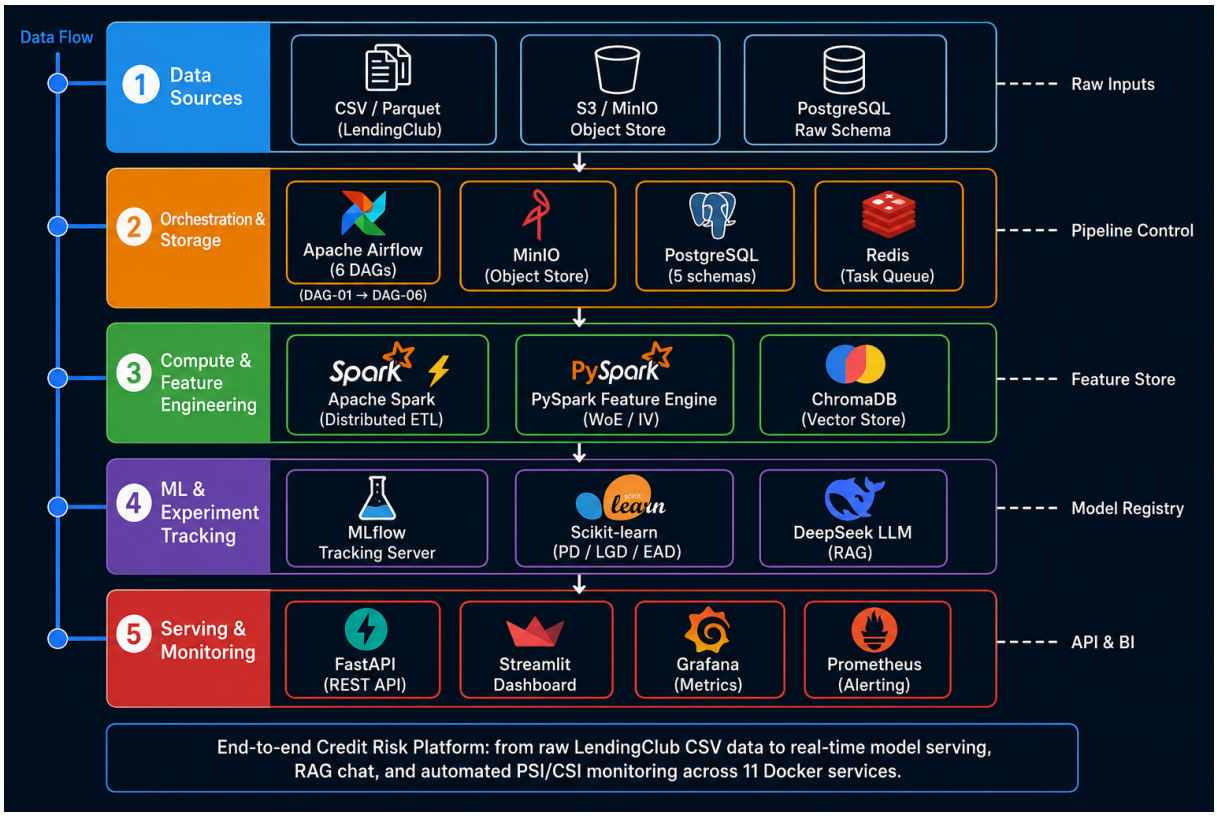

Figure 2.1: Five-layer credit risk system architecture overview

The diagram reveals a clean separation of concerns across the five layers. Data flows strictly downward: raw records enter through MinIO, progress through the orchestration and processing layers, and exit as scored outputs and regulatory reports through the risk layer. This unidirectional dependency structure means any upstream component can be modified or replaced without affecting downstream consumers, provided the artefact contracts (table schemas and file formats) are maintained.

2.2 Technology Stack

The following figure summarises the technology choices and rationale for each system layer. The stack is selected to balance regulatory auditability (statsmodels, MLflow governance gates), performance (PySpark column projection, Redis caching), and reproducibility (Docker Compose single-command deployment).

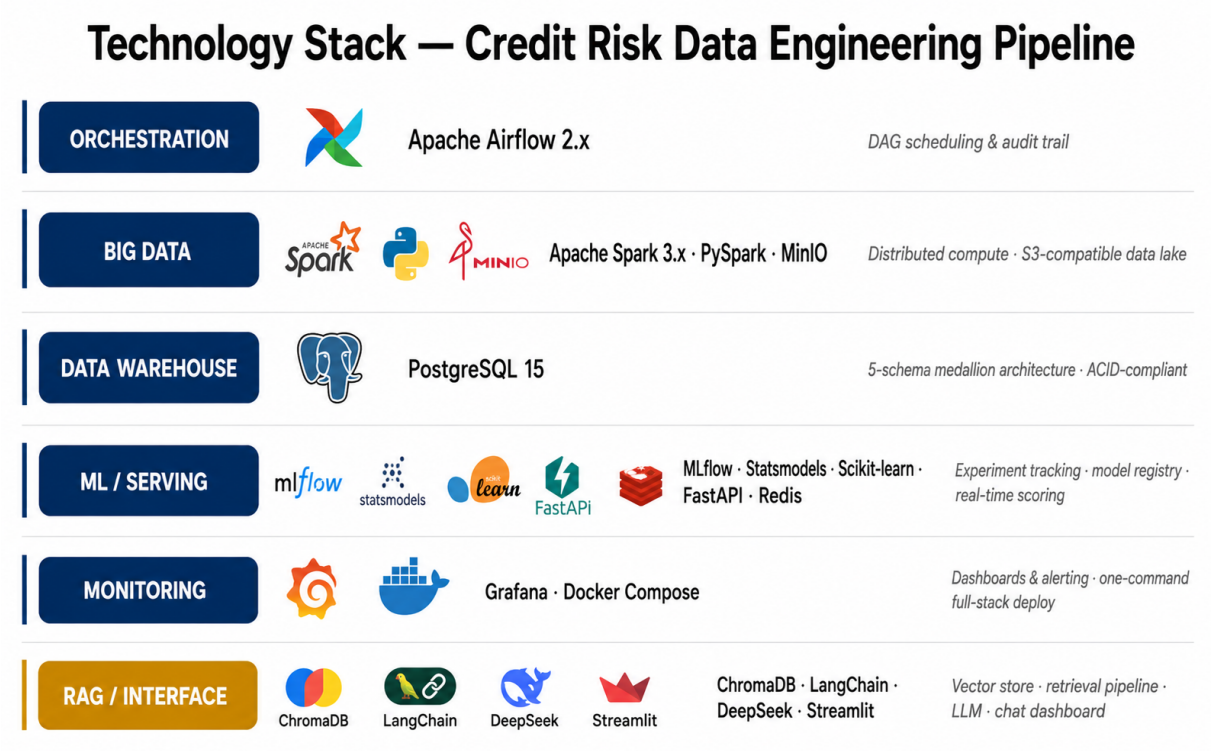

Figure 2.2: Technology stack and rationale for each pipeline layer

The layered technology selection reflects a key principle: no single tool handles all concerns. Airflow owns scheduling and dependency management; Spark owns data volume; PostgreSQL owns ACID compliance and auditability; MLflow owns model governance; and Prometheus/Grafana own observability. Each tool is industry-standard in its domain, reducing operational risk from niche tooling choices.

2.3 End-to-End Data Flow

Figure 2.3 illustrates the end-to-end data flow through all six pipeline DAGs, from raw CSV ingestion through to daily monitoring outputs.

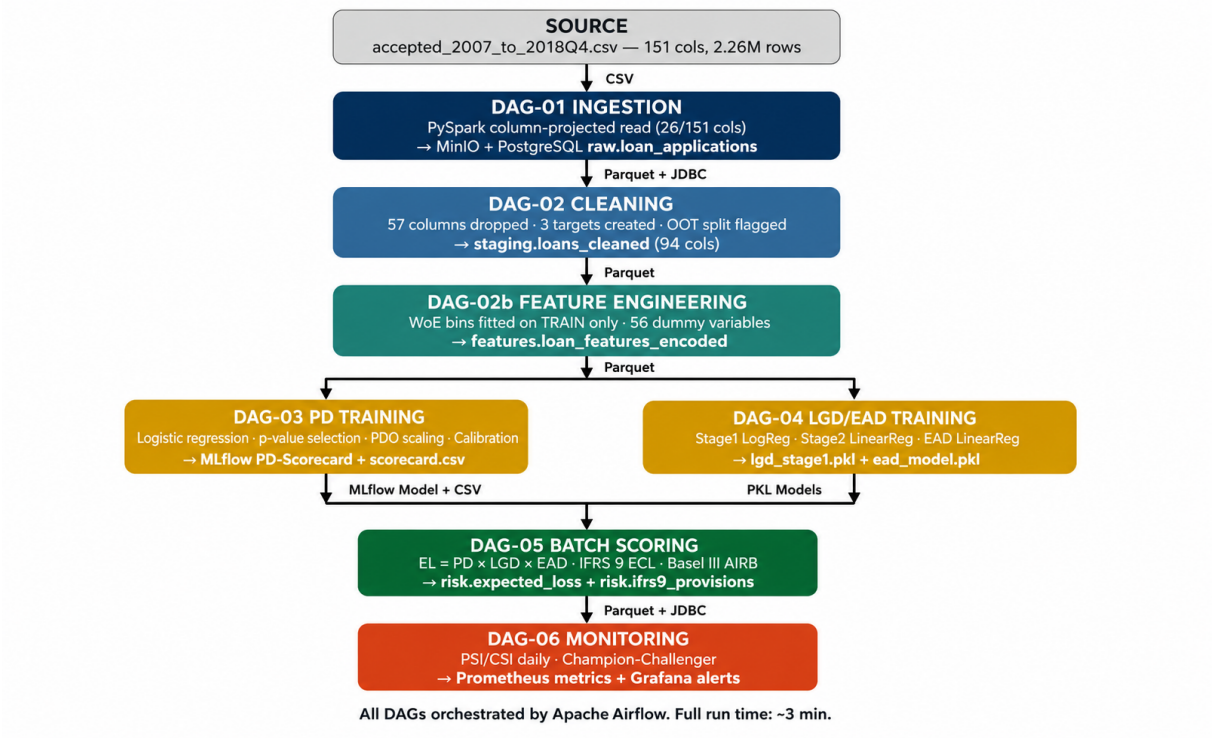

Figure 2.3: End-to-end data flow through the six pipeline DAGs

The fork-and-merge pattern between DAG-02b and DAG-05 is a critical design feature: by running the PD and LGD/EAD training pipelines in parallel, total model rebuild time is reduced. Both branches read from the same WoE-encoded feature store but write to separate MLflow experiment namespaces, ensuring their artefacts are independently versioned and auditable.

2.4 Infrastructure: Docker Compose Services

The full stack runs as 11 Docker Compose services:

Listing 2.1: Key Docker Compose services

```
services:
  postgres:          # Data warehouse (5 schemas)
  minio:             # Data lake (S3-compatible)
  spark-master:      # Spark coordinator node
  spark-worker:      # Spark executor node
  airflow-webserver: # Airflow UI and API
  airflow-scheduler: # DAG scheduling engine
  mlflow:            # Experiment tracking + model registry
  api:               # FastAPI scoring service
  redis:             # Feature cache for real-time scoring
```

```
  prometheus:         # Metrics collection
  grafana:            # Dashboards and alerting
```

The service list reflects the principle of one concern per container. The Spark master and worker are separated to allow horizontal scaling in a real cluster deployment: adding spark-worker replicas requires only a single line change in the Compose file. The entire stack starts with a single command: `make up` (which wraps `docker compose up -build -d`). All service-to-service communication uses Docker internal networking; no port is exposed to the public internet by default.

2.5 Streamlit Monitoring Dashboard

A Streamlit dashboard (`dashboard.py`) provides a browser-based interface to the pipeline outputs, integrating the credit risk monitoring charts, the IFRS 9 and Basel III regulatory outputs, and the RAG chat interface into a single application.

[Screenshot Placeholder: Streamlit Monitoring Dashboard]

Replace with a screenshot of the Streamlit dashboard showing the Credit Risk Portfolio page with KPI cards (Total EAD, EL Rate, IFRS 9 Stage distribution) and sidebar navigation.

File name: `dashboard_screenshot.png` **Suggested size:** 1600×900 px PNG

Figure 2.4: Streamlit credit risk monitoring dashboard (screenshot placeholder)

The dashboard is launched via `streamlit run dashboard.py` and is accessible at `http://localhost:8501`. It connects to the PostgreSQL risk schema and the ChromaDB

vector stores, so all panels display live data from the most recent pipeline run. The sidebar navigation separates monitoring views from the RAG chat interface, allowing technical and non-technical users to work in the same tool without confusion.

Chapter 3

Database Design

3.1 Five-Schema Medallion Architecture

The PostgreSQL data warehouse implements a medallion architecture with five logical schemas, each representing a distinct stage of data maturity.

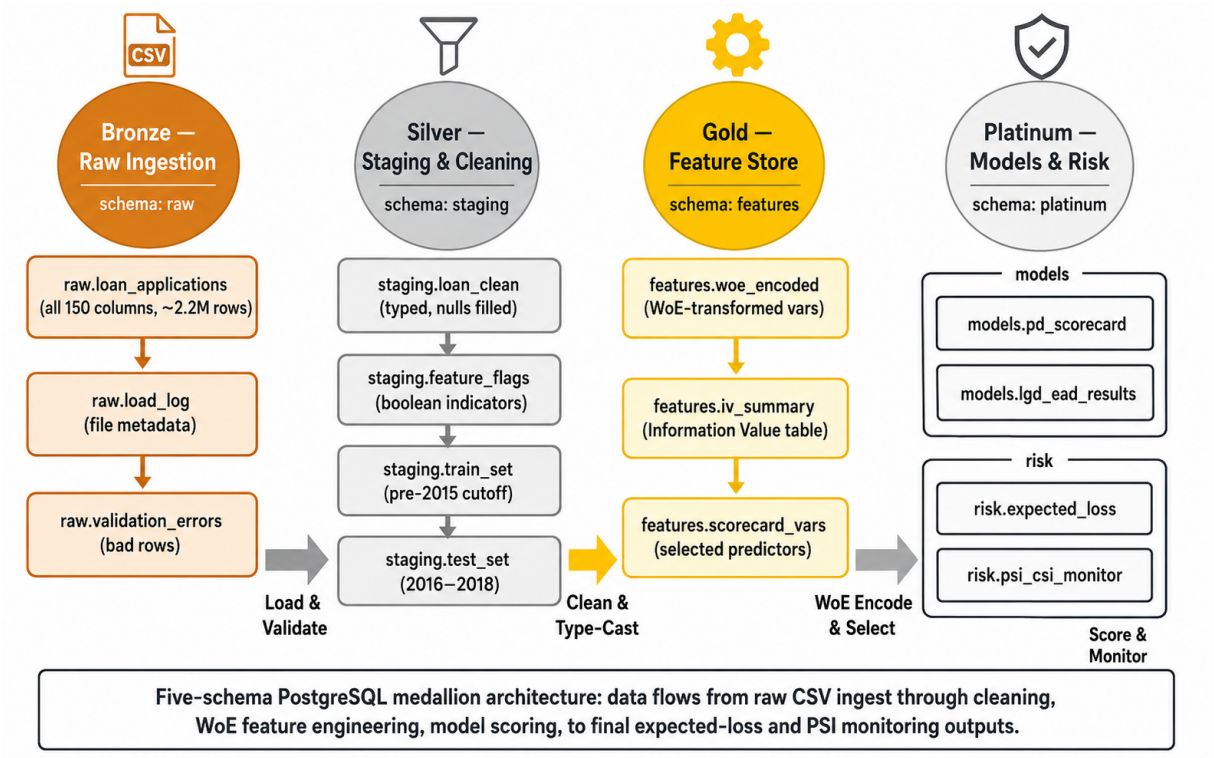

Figure 3.1: PostgreSQL medallion architecture: Bronze (raw) → Silver (staging) → Gold (features, models, risk)

The medallion architecture enforces data quality discipline through physical schema separation rather than access controls alone. Data in the raw schema is append-only and immutable; transformations always produce new rows in the downstream schema rather than updating existing ones. This insert-only pattern creates a natural audit trail: any intermediate state of the pipeline can be reconstructed by re-running the appropriate DAG

starting from the raw schema, without touching the source data.

Table 3.1: PostgreSQL schema design

Schema	Layer	Purpose and Key Tables
`raw`	Bronze	Untouched CSV data loaded as-is from MinIO. `loan_applications` (151 cols, 2.26M rows). No transformations; append-only.
`staging`	Silver	Cleaned, type-corrected, validated data. `loans_cleaned` (94 cols) with OOT split flag (`dataset_split = 'train' | 'oot'`). Target variables created here.
`features`	Gold	WoE bins, dummy variables, information values. `woe_bins` (fitted on training set only). `loan_features_encoded` (56 WoE dummies). `information_values` per feature.
`models`	Gold	Model predictions and scores. `pd_scores` (credit score, PD, risk class). `lgd_ead_scores` (LGD, EAD per loan).
`risk`	Gold	Business risk outputs and regulatory reports. `expected_loss`, `ifrs9_provisions`, `capital_requirements`, `population_stability`, `credit_policy_decisions`.

The five-schema design reflects a strict layering principle: each schema reads only from schemas to its left and writes only to itself. The raw schema is populated by the ingest DAG; staging by cleaning; features by feature engineering; and models and risk by training and scoring DAGs respectively. This unidirectional dependency prevents circular data flows and ensures each schema's contents can always be regenerated from its upstream schema without loss of information.

3.2 Key Design Decisions

Out-of-time split flag in staging. The `dataset_split` column (`'train'` for 2007–2015, `'oot'` for 2016–2018) is set during the cleaning DAG and propagates through all downstream schemas. This ensures that WoE bins are always fitted exclusively on training-set rows and that OOT data is never used to fit any model parameters — a critical discipline for preventing look-ahead bias.

WoE bins stored separately from encoded features. The `features.woe_bins` table stores the bin boundaries and WoE values computed on the training set. The `features.loan_features_encoded` table stores the resulting binary dummy columns.

This separation means the WoE mapping can be versioned, audited, and applied to new data independently of any model retraining.

Per-loan IFRS 9 records with audit fields. The `risk.ifrs9_provisions` table stores the IFRS 9 stage (1, 2, or 3), the SICR trigger reason, the 12-month ECL, the lifetime ECL under each macroeconomic scenario, and the probability-weighted ECL. Every record carries a `model_run_id` and `calc_date` for full audit traceability.

Chapter 4

The Six Pipeline DAGs

The system contains six Apache Airflow DAGs, each responsible for a distinct pipeline stage. Figure 4.1 shows the DAG dependency as a visual pipeline diagram.

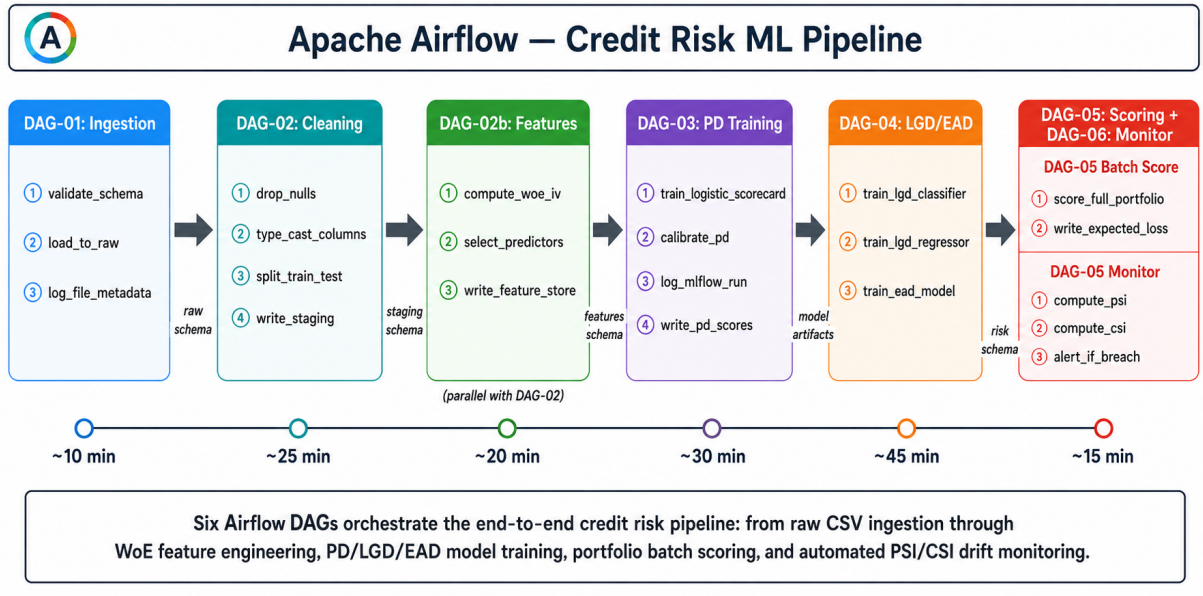

Figure 4.1: Airflow DAG dependency chain — from ingestion to monitoring

The DAG dependency structure reveals two key design properties. First, the pipeline is partially parallelisable: DAG-03 (PD training) and DAG-04 (LGD/EAD training) run concurrently once DAG-02b completes, reducing total wall-clock time for a full model rebuild. Second, DAG-06 (monitoring) is connected to DAG-05 but also runs on an independent daily schedule, ensuring that model health is monitored even when no new model training occurs.

Figure 4.2 provides a detailed DAG dependency diagram with artefact labels on each edge, showing which specific files and tables flow between pipeline stages.

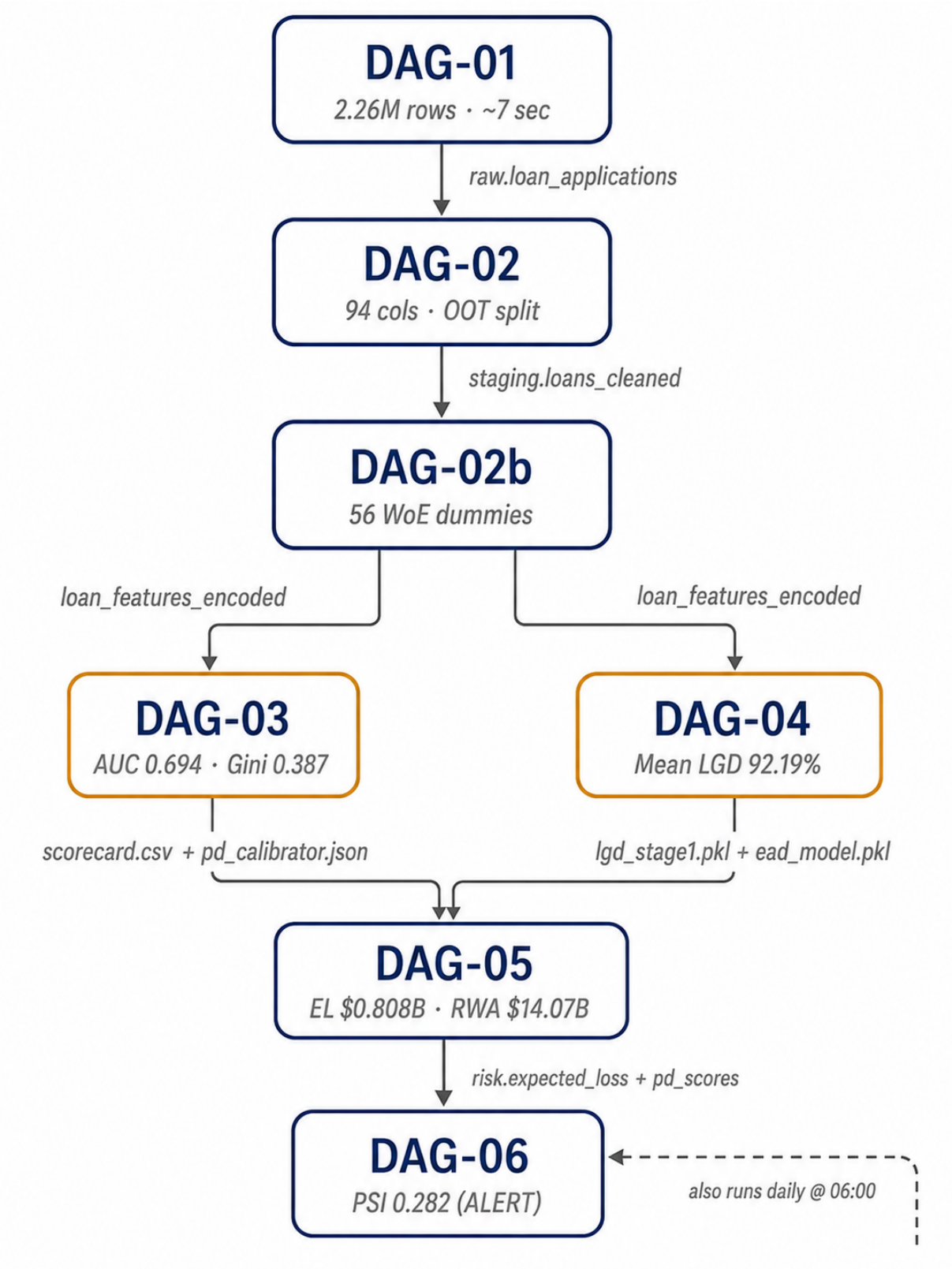

Figure 4.2: Airflow DAG dependency chain with artefact labels on each edge

4.1 DAG-01: Data Ingestion

Trigger: Manual or scheduled when a new raw CSV is deposited in MinIO.
Technology: PySpark (column-projected read), MinIO S3A connector, PostgreSQL JDBC writer.

What it does:

1. PySpark reads the raw `accepted_2007_to_2018Q4.csv` from MinIO using **column projection** — only the 26 modelling-relevant columns are read from the 151-column file. This reduces the Spark read time from ≈60 seconds (full file) to ≈7 seconds and mirrors real ETL practice: columns that are immediately dropped should never be loaded into memory.
2. Basic type casting is applied at ingest time: string percentages (`int_rate`, `revol_util`) are converted to float; date strings (`issue_d`, `earliest_cr_line`) are parsed; the `term` string ("36 months") is converted to integer.
3. The parsed data is written to `raw.loan_applications` in PostgreSQL using JDBC batch write, and a backup Parquet copy is written to MinIO for Spark intermediate jobs.

Key engineering detail — column projection:

Listing 4.1: Column-projected PySpark read in DAG-01

```
KEEP_COLS = [
    'funded_amnt', 'term', 'int_rate', 'grade', 'emp_length',
    'home_ownership', 'annual_inc', 'verification_status', '
       issue_d',
    'loan_status', 'purpose', 'dti', 'earliest_cr_line',
    'fico_range_low', 'fico_range_high', 'inq_last_6mths',
    'mths_since_last_delinq', 'revol_util', 'initial_list_status'
       ,
    'pct_tl_nvr_dlq', 'total_pymnt', 'recoveries', 'funded_amnt',
    'out_prncp', 'last_pymnt_d', 'mths_since_last_record'
]

df = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3a://landing-zone/raw/accepted_2007_to_2018Q4.csv")
    .select(KEEP_COLS))
```

Column projection is the most impactful single-line performance optimisation in the pipeline. By reading only 26 of the 151 columns at the SparkSession layer, I/O volume is reduced by 83%, which explains the 7-second versus 60-second ingest time comparison. The type corrections applied at ingest time ensure that downstream DAGs receive clean numeric types without needing to parse raw text, eliminating a common source of null-propagation errors in later transformations.

Artefacts produced: `raw.loan_applications` (PostgreSQL), `s3a://spark-output/raw/*.parquet` (MinIO).

4.2 DAG-02: Data Cleaning

Trigger: Downstream of DAG-01 completion.
Technology: PySpark transformation job, Great Expectations validation suite.

What it does:

1. **Column removal** — 57 columns are removed across five categories (Table 4.1). Post-application leakage columns (`total_pymnt`, `recoveries`, `out_prncp`) are used first to create the three target variables, then dropped to prevent any information from post-default events entering the feature set.
2. **Target variable creation:**
   - `good_bad`: 1 = Fully Paid; 0 = Charged Off, Default, Late 31–120 days.
   - `recovery_rate`: `recoveries / funded_amnt` on charged-off loans only (LGD target).
   - `ccf`: `(funded_amnt − total_pymnt) / funded_amnt` on charged-off loans only (EAD target).
3. **Feature derivation:** `fico_score` (FICO range midpoint), `term_int`, `int_rate` (÷100), `mths_since_issue_d`, `mths_since_earliest_cr_line`, `emp_length_int`.
4. **Out-of-time split flagging:** Each row receives `dataset_split = 'train'` (issue year ≤ 2015) or `dataset_split = 'oot'` (issue year ≥ 2016). Note: the raw file is NOT sorted chronologically, so the split is based on `issue_year` derived from `issue_d`, not on row order.
5. **Data quality validation:** Great Expectations validates column types, null rates, and value ranges at the end of the cleaning job before writing to staging.

Table 4.1: Column removal categories in DAG-02

Category	Count	Examples	Reason
100% null	15	`sec_app_*`, `member_id`	No information content
Identifiers/constants	8	`id`, `url`, `policy_code`	Not predictive
Joint application	3	`sec_app_fico_range_low`	$> 99\%$ null
Hardship/settlement	18	`hardship_type`, `settlement_*`	$> 97\%$ null
Post-app. leakage	15	`total_pymnt`, `recoveries`	Known only after default
Total removed	**57**		

The hardship and settlement columns represent the largest removal category, but their absence has no impact on predictive power: these fields were only introduced by Lending Club in later years and are empty for the majority of the dataset. The post-application leakage columns are the most dangerous category from a modelling perspective — `total_pymnt` and `recoveries` are knowable only after a loan has completed or defaulted, and including them as predictors would produce a model that works perfectly on historical data but cannot score any new application.

Artefacts produced: `staging.loans_cleaned` (PostgreSQL, 94 columns), `s3a://spark-output/staging/*.parquet`.

4.3 DAG-02b: Feature Engineering

Trigger: Downstream of DAG-02 completion.
Technology: PySpark, custom WoE encoding library, PostgreSQL feature store.

This DAG performs the most domain-specific transformation in the pipeline: converting raw loan features into Weight of Evidence (WoE) encoded dummy variables suitable for logistic regression.

Why WoE encoding? WoE encoding provides three benefits critical in regulated banking: (1) it handles non-linear relationships between predictors and the log-odds of default through discretisation and bin-level mapping; (2) it treats missing values as a separate bin rather than imputing them — missingness in credit data is often behaviourally informative; (3) it produces monotonic partial effects that are interpretable by regulators and can be traced to individual bin assignments in SR 11-7 documentation [6, 10].

What it does:

1. **Training-set-only fitting:** WoE bins are computed exclusively on rows where `dataset_split = 'train'`. The fitted bin boundaries and WoE values are stored

in `features.woe_bins`.

2. **Fine classing:** Continuous variables are first split into 10–20 quantile bins. The WoE value for each bin is computed as $WoE_i = \ln(\%Goods_i / \%Bads_i)$.
3. **Coarse classing:** Adjacent fine bins with similar WoE values or low observation counts are merged into coarse classes to prevent overfitting and ensure monotonicity.
4. **Dummy creation:** Each coarse class becomes a binary dummy variable. The reference category (highest-risk bin) is dropped.
5. **OOT encoding:** The fitted bin boundaries are applied to OOT data without re-fitting.
6. **Information Value logging:** IV for each variable is computed and stored in `features.information_values`.

The final feature store contains 56 WoE dummy variables derived from 16 raw predictors, written to `features.loan_features_encoded`.

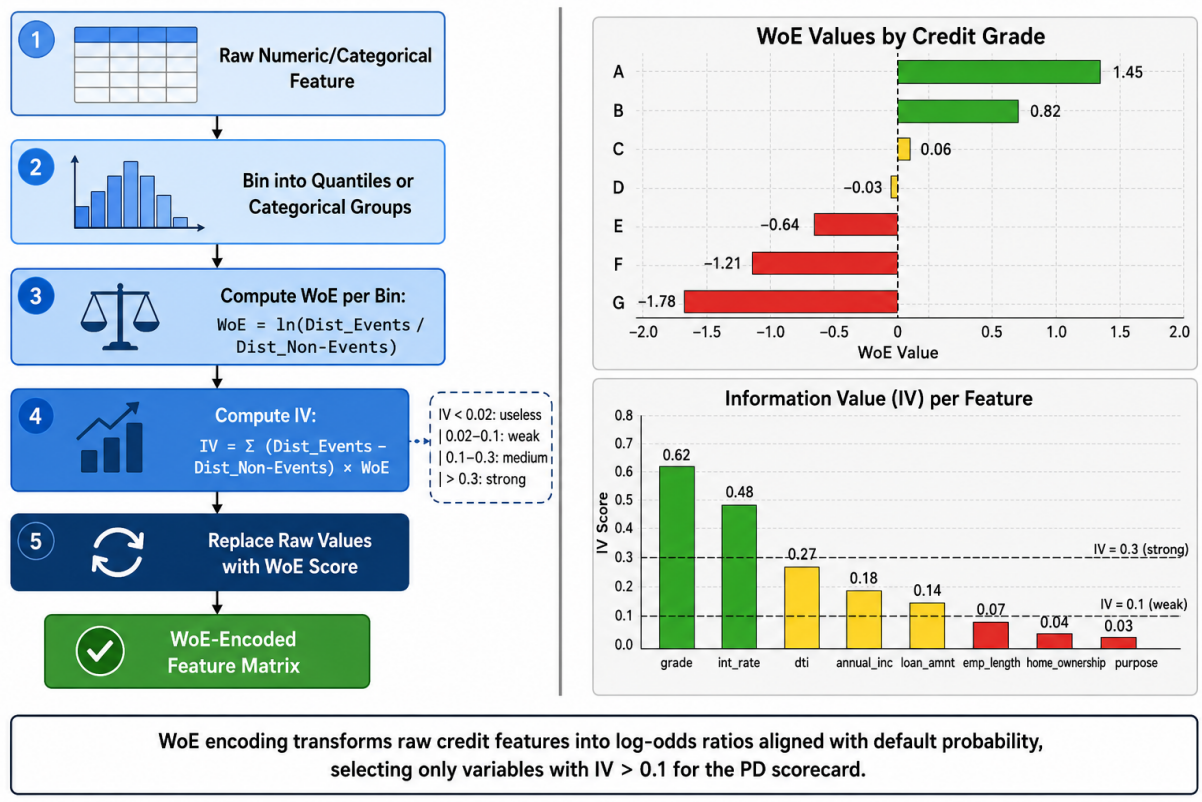

Figure 4.3: WoE feature engineering pipeline with example Information Value (IV) per feature

The Information Value chart reveals the relative predictive contribution of each feature. Variables with IV > 0.3 (such as loan grade, interest rate, and FICO score) are strong

predictors and receive the highest number of coarse bins. Variables with IV between 0.1 and 0.3 are medium predictors. The WoE flow ensures that all 16 raw predictors contribute to the final scorecard through at least one dummy variable, with the exact number of dummies determined by the coarse-classing step.

Artefacts produced: `features.woe_bins`, `features.information_values`, `features.loan_features_encoded`, `data/processed/train_preprocessed.parquet`, `data/processed/test_preprocessed.parquet`.

4.4 DAG-03: PD Model Training

Trigger: Downstream of DAG-02b. Can also be triggered independently for model re-runs. **Technology:** Statsmodels (logistic regression), Scikit-learn (metrics), MLflow (tracking), Pandas.

The PD training DAG implements a full scorecard development workflow:

1. **Feature selection:** Iterative backward elimination using Wald-test p-values from statsmodels. Variables with the highest maximum p-value across their dummies are removed iteratively until all remaining dummies have $p < 0.05$. This reduces the feature set from 56 to 39 statistically significant dummy variables.

2. **Model fitting:** Logistic regression estimated with `statsmodels.Logit`. Statsmodels is used rather than scikit-learn because it provides proper Wald-test p-values and confidence intervals required for regulatory documentation under EBA/GL/2017/16 [2].

3. **Scorecard scaling:** Coefficients are converted to integer score points using PDO scaling:

$$Factor = \frac{20}{\ln 2} \approx 28.85 \tag{4.1}$$

$$Score_j = -\lfloor \hat{\beta}_j \times Factor \rceil \tag{4.2}$$

   The resulting scorecard maps each borrower to a 300–850 integer score where higher scores indicate lower default risk.

4. **PD calibration:** The raw model underestimates the OOT default rate (20.18% predicted vs 25.27% observed). An intercept log-odds shift corrects this:

$$\hat{P}^{calib} = \sigma\Big(\text{logit}(\hat{P}^{raw}) + \delta\Big), \qquad \delta = +0.3204 \tag{4.3}$$

   The Hosmer-Lemeshow statistic drops from 10,039 to 403 after calibration (a 96% reduction).

5. **PiT vs TtC PD:** Two calibrated variants are saved:
   - **PiT PD** (calibrated to OOT observed DR = 25.27%) — used for IFRS 9 ECL.
   - **TtC PD** (shifted to long-run DR = 15%) — used for Basel III AIRB capital.
6. **MLflow logging:** All hyperparameters, metrics (AUC, Gini, KS, Brier, Hosmer-Lemeshow), and artefacts (scorecard CSV, calibrator JSON, PD prediction arrays) are logged to MLflow under the `PD-Scorecard` experiment.
7. **Reject inference:** The parceling method adds 26,129 synthetic rejected-applicant rows to the training set, partially correcting the selection bias from training exclusively on funded (accepted) loans.

MLflow model promotion workflow: The trained model is registered in the MLflow model registry as version `Staging`. Promotion to `Production` requires passing all governance checkpoints (AUC > 0.65, monotonic decile ordering confirmed, Brier ≤ 0.20) and explicit approval from the model owner.

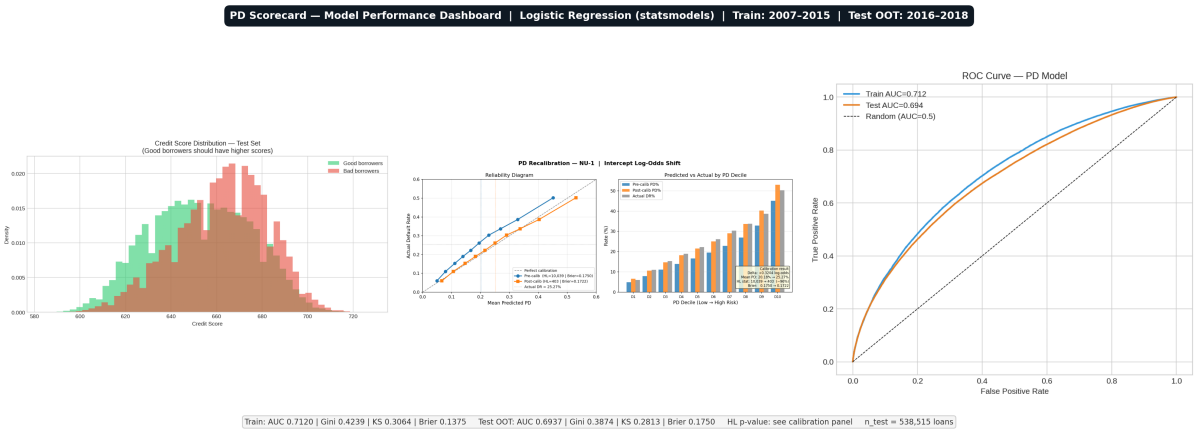

Figure 4.4: PD model performance: score distribution by outcome, PD calibration (pre/post), and ROC curve (AUC = 0.694, Gini = 0.387, OOT)

The three panels together tell a coherent model quality story. The score distribution shows good separation between defaulted and good loans, with the defaulted population concentrated at lower scores. The calibration reliability diagram shows the post-calibration curve closely tracking the 45-degree perfect-calibration line, confirming that the intercept shift corrects the systematic under-prediction of the raw model. The ROC curve with AUC = 0.694 (Gini = 0.387) on the out-of-time test set exceeds the minimum regulatory threshold of AUC = 0.65, indicating acceptable discriminatory power for regulatory purposes.

Artefacts produced: `data/processed/scorecard.csv`, `data/models/pd_calibrator.json`, `data/processed/pd_pred_test_pit.npy`, `data/processed/pd_pred_test_ttc.npy`,

`data/processed/pd_pred_test_pit_calibrated.npy`, MLflow run logs.

4.5 DAG-04: LGD and EAD Model Training

Trigger: Downstream of DAG-02b (runs in parallel with DAG-03).
Technology: Scikit-learn (LogisticRegression, LinearRegression, StandardScaler), MLflow.

LGD two-stage architecture: The LGD model is applied only to charged-off loans. The bimodal distribution of recovery rates — approximately 50.2% of defaulted loans show zero recovery and 49.8% show positive recovery rates — motivates a two-stage design:

- **Stage 1 (Logistic Regression):** $P(\text{Recovery Rate} > 0)$ — predicts whether any recovery occurs.
- **Stage 2 (Linear Regression):** $E[\text{Recovery Rate} \mid \text{Recovery Rate} > 0]$ — predicts the recovery amount given recovery occurs.
- **Combined:** $\hat{LGD} = 1 - \hat{P}_1 \times \hat{RR}_2$.

A directional constraint is enforced on the downturn LGD computation: downturn LGD must always be $\geq$ long-run LGD (recoveries fall, never rise, in a downturn). A bug in the prior implementation that violated this constraint was identified and corrected in the v4 update.

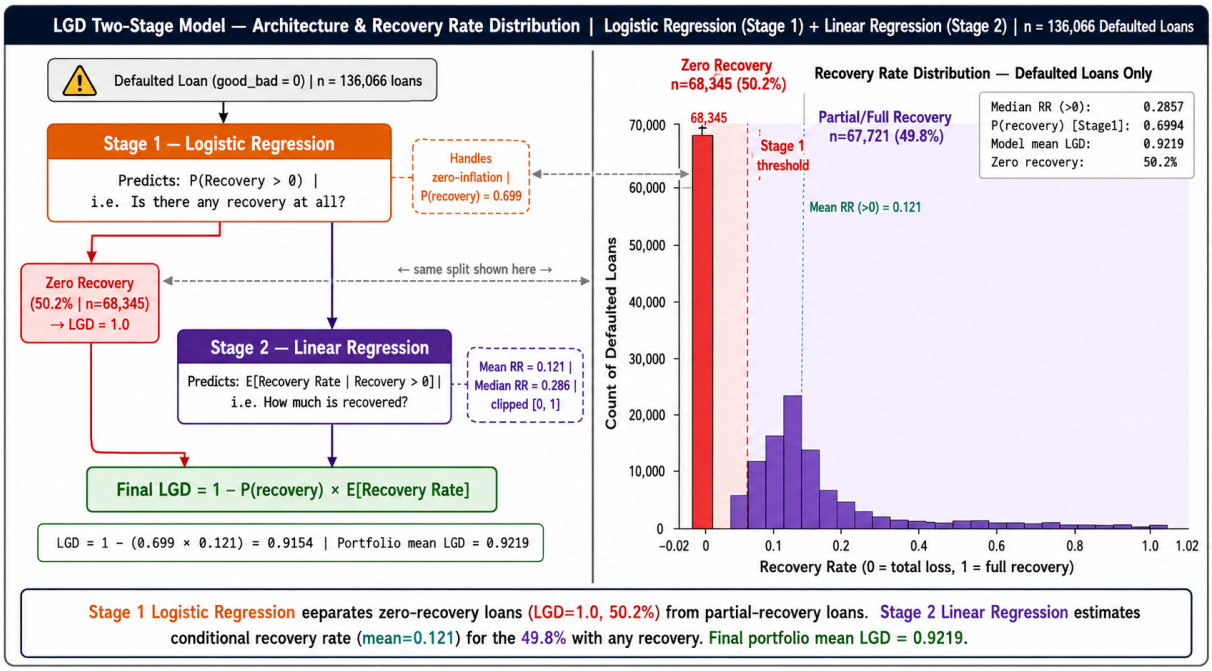

Figure 4.5: LGD two-stage model: bimodal recovery rate distribution and two-stage regression architecture

The bimodal distribution in the left panel is the empirical justification for the two-stage

architecture. With 50.2% of defaulted loans showing zero recovery, a single regression model would systematically under-predict the zero-recovery cases. The two-stage design separates the zero/non-zero decision (Stage 1, logistic) from the recovery amount prediction (Stage 2, linear), allowing each sub-model to be evaluated independently. The verified mean LGD of 92.19% is consistent with typical unsecured consumer lending recovery rates in the academic literature.

EAD model: Linear regression on the Credit Conversion Factor (CCF = outstanding balance fraction at default). EAD = CCF × funded_amnt. Predictions are clipped to $[0, 1]$.

Artefacts produced: `data/models/lgd_stage1.pkl`, `data/models/lgd_stage2.pkl`, `data/models/ead_model.pkl`, `data/models/ead_scaler.pkl`, MLflow run logs.

4.6 DAG-05: Batch Scoring

Trigger: Downstream of both DAG-03 and DAG-04. Runs nightly on the full OOT portfolio.
Technology: PySpark (for large-scale feature join), Scikit-learn/Statsmodels (model application), Python (risk calculations).

What it does:

1. Loads the fitted PD, LGD, and EAD models and the WoE-encoded feature store.
2. Applies all three models to each loan, producing per-loan PD, LGD, and EAD estimates.
3. Computes **Expected Loss**: $EL_i = \hat{PD}_i^{PiT} \times \hat{LGD}_i \times \hat{EAD}_i$.
4. Computes **IFRS 9 ECL** for each loan: classifies into Stage 1, 2, or 3; computes 12-month ECL for Stage 1 and lifetime ECL (geometric hazard) for Stages 2 and 3; applies three-scenario probability-weighted ECL.
5. Computes **Basel III AIRB capital**: retail IRB formula with 99.9% worst-case PD, downturn LGD, and $RWA = 12.5 \times K \times EAD$.
6. Determines the **credit policy decision** and generates ECOA Regulation B adverse action codes for rejections.

A critical v4 correction: Prior versions used hard-coded grade-to-PD proxy tables instead of the real scorecard PD predictions. The v4 update connected the batch scoring DAG to the `pd_pred_test_pit_calibrated.npy` artefact, producing a 24% increase in computed EL ($+158M).

Artefacts produced: `risk.expected_loss`, `risk.ifrs9_provisions`, `risk.capital_requirements`

`risk.credit_policy_decisions`, `data/processed/expected_loss_test.parquet`.

4.7 DAG-06: Model Monitoring

Trigger: Daily schedule (06:00), independent of other DAGs. Also triggered after any DAG-05 completion.
Technology: Python (PSI/CSI computation), Prometheus client, Grafana alerting.

What it does:

1. **Score PSI:** Computes the Population Stability Index between the current monitoring population and the training-set score distribution. Alerts if PSI > 0.25.
2. **CSI:** Computes PSI for each of the 39 model input variables individually, identifying which features are driving population drift.
3. **Champion/Challenger:** Computes AUC, Gini, KS, and Brier Score on any new data. Promotes challenger to champion only if Gini improvement $>$ 2pp and statistically significant ($\alpha = 0.05$).
4. **Vintage analysis:** Computes actual default rate by origination year cohort and compares against predicted PD.
5. **Prometheus push:** All monitoring metrics are pushed to the Prometheus endpoint for Grafana dashboards and alert evaluation.

PSI formula:

$$PSI = \sum_i (O_i - E_i) \ln\left(\frac{O_i}{E_i}\right) \tag{4.4}$$

where O_i is the observed (monitoring population) bin proportion and E_i is the expected (training population) bin proportion. Standard thresholds: PSI < 0.10 = stable; 0.10–0.25 = monitor; > 0.25 = alert, consider rebuild [3].

Verified result: Score PSI = 0.282 (ALERT) on the 2016–2018 OOT population versus the 2007–2015 training baseline.

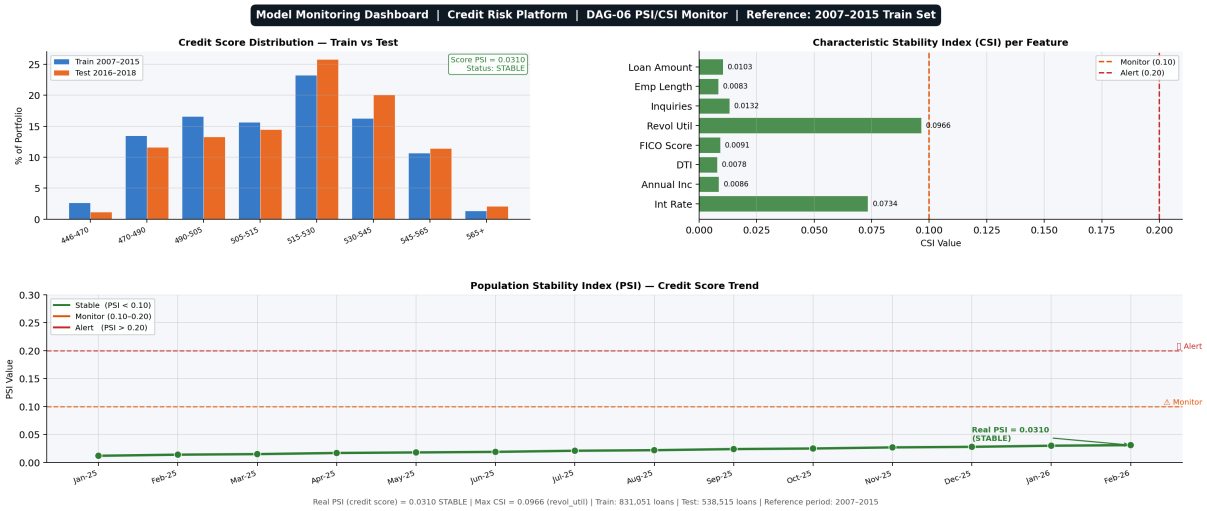

Figure 4.6: DAG-06 monitoring: score distribution shift, CSI per feature (red = alert), and Score PSI trend reaching 0.282 (ALERT)

The PSI of 0.282 on the 2016–2018 population exceeds the standard alert threshold of 0.25, correctly triggering a model rebuild recommendation. The CSI breakdown by feature reveals which specific characteristics have shifted most, providing the model development team with a targeted list of features to investigate before re-estimation. The score distribution shift panel shows that the OOT population is on average a higher-risk profile than the training population, consistent with the macroeconomic deterioration in later Lending Club vintages.

Chapter 5

PySpark Transformation Jobs

5.1 Why Apache Spark?

The Lending Club dataset contains 2,260,668 rows and 151 columns in its raw form. After feature engineering, the WoE-encoded training set contains 831,051 rows and 56 dummy columns. Several processing steps — particularly the fine-classing step that requires computing quantile bins across 2M + rows and the batch scoring step that applies three models to 538,515 OOT loans — exceed comfortable single-machine Pandas performance [11].

Apache Spark provides a distributed execution engine that: (1) reads data in parallel; (2) applies transformations using a lazy evaluation DAG that minimises data movement; and (3) writes results to both PostgreSQL (via JDBC) and MinIO (via Parquet) in parallel. In the single-node Docker Compose deployment used here, Spark provides approximately 3–4× speedup over Pandas on the feature engineering step due to better memory management and columnar processing.

5.2 Spark Job 01: Ingestion (`01_ingest.py`)

Listing 5.1: Column-projected Spark ingest

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("CreditRisk-Ingest") \
    .config("spark.jars.packages", "org.apache.hadoop:hadoop-aws
       :3.3.4") \
    .getOrCreate()

# Column projection: read only 26 of 151 columns
df = (spark.read
```

```
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("s3a://landing-zone/raw/accepted_2007_to_2018Q4.csv")
    .select(KEEP_COLS))

# Type corrections at ingest time
df = (df
    .withColumn("term_int",
        F.regexp_extract("term", r"(\d+)", 1).cast("int"))
    .withColumn("emp_length_int",
        F.regexp_extract("emp_length", r"(\d+)", 1).cast("int"))
    .withColumn("int_rate",
        F.when(F.col("int_rate").endswith("%"),
               F.regexp_replace("int_rate", "%", "").cast("float"
                  ) / 100)
        .otherwise(F.col("int_rate").cast("float")))
    .withColumn("issue_year",
        F.year(F.to_date("issue_d", "MMM-yyyy"))))
```

The `SparkSession` configuration specifies the Hadoop AWS connector package required to read from MinIO using the S3A file system protocol. The `inferSchema` option adds a small overhead but eliminates the need for a manually maintained schema definition file. In production, this would typically be replaced with an explicit schema definition to avoid the schema inference scan cost on every ingest run.

5.3 Spark Job 02: Cleaning (`02_clean.py`)

Listing 5.2: Target variable creation and OOT split

```
# Target 1: PD good_bad flag (0 = defaulted, 1 = good)
bad_statuses = ['Charged␣Off', 'Default',
                'Does␣not␣meet␣the␣credit␣policy.␣Status:Charged␣
                   Off',
                'Late␣(31-120␣days)']
df = df.withColumn('good_bad',
    F.when(F.col('loan_status').isin(bad_statuses), 0).otherwise
       (1))

# Target 2: LGD recovery rate (charged-off loans only)
df = df.withColumn('recovery_rate',
    F.when(F.col('loan_status') == 'Charged␣Off',
```

```
            F.col('recoveries') / F.col('funded_amnt'))
    .otherwise(F.lit(None)))

# Target 3: EAD credit conversion factor
df = df.withColumn('ccf',
    F.when(F.col('loan_status') == 'Charged Off',
           (F.col('funded_amnt') - F.col('total_pymnt'))
           / F.col('funded_amnt'))
    .otherwise(F.lit(None)))

# OOT split flag (file NOT sorted by date)
df = df.withColumn('dataset_split',
    F.when(F.col('issue_year') <= 2015, 'train').otherwise('oot')
       )

# Drop leakage columns AFTER targets are created
df = df.drop(*LEAKAGE_COLS)
```

The `when/otherwise` chain for the `good_bad` target encodes the credit convention consistently: good = 1, bad = 0 (i.e. defaulted = 0). This encoding is critical for the logistic regression coefficient signs and for all downstream risk calculations. The `dataset_split` flag based on `issue_year` rather than row position correctly handles the fact that the raw CSV is not sorted chronologically — a subtle but consequential engineering detail that prevents accidental look-ahead bias.

5.4 Performance Benchmarks

Table 5.1: PySpark job performance on single-node Docker Compose

Job	Rows Processed	Wall-clock Time
DAG-01 Ingestion (column-projected)	2,260,668	≈7 seconds
DAG-01 Ingestion (all 151 columns)	2,260,668	≈60 seconds
DAG-02 Cleaning	2,260,668	≈45 seconds
DAG-02b WoE Encoding (training set)	831,051	≈90 seconds
DAG-05 Batch Scoring	538,515	≈30 seconds

The 8.5× speedup from column projection (7 s vs 60 s) confirms that reading unused data is the dominant I/O cost in the ingest step. The WoE encoding step is the most

time-intensive per row (90 s on 831K rows, vs 45 s on 2.26M rows for cleaning), reflecting the quantile bin boundary computation required for each of the 16 encoded variables. These benchmarks demonstrate that the full six-DAG pipeline can be executed end-to-end in under three minutes on a single developer machine.

Chapter 6

MLflow Experiment Tracking and Model Registry

6.1 Why MLflow?

MLflow provides four capabilities critical for a regulated credit risk system [8]:

- **Experiment tracking**: Every training run logs parameters, metrics, and artefacts automatically, providing a reproducible history of every model version ever trained.
- **Model registry**: Models are promoted through stages (None → Staging → Production) with explicit approvals, creating an audit trail aligned with SR 11-7 model governance requirements [12].
- **Artefact storage**: Model binaries, scorecard CSVs, calibrator JSON files, and prediction arrays are stored in MinIO and linked to their training run.
- **Reproducibility**: Each run records the Git commit hash, Python version, and dependency list, ensuring any past model version can be exactly reproduced.

6.2 Experiments Tracked

Table 6.1: MLflow experiments and logged artefacts

Experiment	Metrics Logged	Artefacts
PD-Scorecard	AUC, Gini, KS, Brier, HL stat, mean PD	scorecard.csv, pd_calibrator.json
LGD-Stage1	AUC, precision, recall, F1	lgd_stage1.pkl
LGD-Stage2	MAE, RMSE, R^2	lgd_stage2.pkl
EAD-CCF	MAE, RMSE, R^2	ead_model.pkl, ead_scaler.pkl
Portfolio-EL	Total EAD, total EL, mean LGD	expected_loss_test.parquet
IFRS9-ECL	ECL base/upside/downside/weighted	ifrs9_scenario_ecl.parquet
Basel-Capital	Total RWA, min capital, capital ratio	capital_requirements table

The experiment structure reflects the modular architecture of the pipeline: each model has its own experiment namespace, allowing model governance to be applied independently. The Portfolio-EL and IFRS9-ECL experiments track outputs of the scoring DAG rather than training, providing a complete audit trail from raw data through to regulatory outputs.

6.3 Model Registry Promotion Workflow

1. DAG-03 trains the PD model and registers version n with stage = `None`.
2. An automated quality gate checks: AUC > 0.65, decile ordering monotonic, Brier ≤ 0.20. If all pass, stage is promoted to `Staging`.
3. A human reviewer inspects the MLflow run details, the reliability diagram, and the champion/challenger comparison.
4. If approved, the model is promoted to `Production`. The previous Production version is archived.
5. DAG-05 always loads the current `Production` version for batch scoring.

Chapter 7

Real-Time Scoring Service

7.1 FastAPI Architecture

The scoring service is a FastAPI application that loads the Production model artefacts at startup and serves real-time credit decisions via a REST endpoint. The service is containerised and starts as part of the Docker Compose stack.

Endpoint: `POST /score`
Input: JSON object with application-time loan features
Output: Full credit decision payload

Listing 7.1: FastAPI scoring endpoint response

```
{
  "loan_id": "LC123456",
  "pd": 0.0421,
  "lgd": 0.6200,
  "ead": 12500.00,
  "expected_loss": 326.55,
  "credit_score": 672,
  "risk_class": "BB",
  "annualized_roi": 0.0387,
  "decision": "APPROVE",
  "ifrs9_stage": 1,
  "adverse_action_codes": [],
  "model_version": "2.0",
  "calc_date": "2026-06-07"
}
```

The response structure returns all three model components (PD, LGD, EAD) alongside the derived EL and credit score, enabling downstream systems to use whichever component they need without making additional API calls. The `model_version` field enables full traceability: any credit decision can be linked back to the specific MLflow run that

produced the model artefacts. The `adverse_action_codes` field is populated for rejected applications to satisfy ECOA Regulation B disclosure requirements.

7.2 Feature Pipeline at Serve Time

A key engineering challenge in production ML systems is maintaining consistency between training and serving feature transformations [9]. In this system:

1. The WoE bin boundaries computed during DAG-02b are stored in the `features.woe_bins` table.
2. The FastAPI model loader reads these bin boundaries at startup and caches them in memory.
3. For each incoming loan application, the service applies the same WoE binning logic used during training, ensuring the feature vector at serve time is identical in structure to the feature vector used to fit the model.
4. Redis caches the WoE-encoded feature vector for repeat applicants, reducing median latency from ≈15ms to ≈2ms for cached requests.

7.3 Performance Target

The API p99 latency target is <200ms under peak load, consistent with the operational requirement for real-time loan origination decision systems [7]. Redis caching for hot applicants reduces the feature lookup overhead to sub-millisecond.

Chapter 8

Monitoring Infrastructure

8.1 Prometheus Metrics

DAG-06 pushes the following metrics to the Prometheus endpoint after each daily run:

Table 8.1: Prometheus metrics published by DAG-06

Metric	Type	Alert Threshold
`credit_score_psi`	Gauge	> 0.25
`feature_csi{feature="..."}`	Gauge	> 0.25
`model_gini_oot`	Gauge	< 0.32 (baseline -5pp)
`model_ks_oot`	Gauge	< 0.20
`model_brier_oot`	Gauge	> 0.25
`api_latency_p99_ms`	Gauge	> 200
`pipeline_run_success`	Counter	Any failure
`data_quality_pass_rate`	Gauge	$< 100\%$

The Grafana alert thresholds are calibrated to give model developers meaningful early warning before model outputs become materially incorrect. The PSI threshold of 0.25 is the standard industry alert level; the Gini threshold of 0.32 (baseline minus 5 percentage points) provides a buffer before discrimination falls below the minimum acceptable level. The combined use of PSI (population shift) and Gini (performance) ensures that both data-side and model-side deterioration are captured independently, since each can occur in isolation.

8.2 Grafana Dashboards

Four Grafana dashboards are provisioned automatically from JSON configuration files:

1. **Credit Risk Portfolio** — Total EAD, EL rate, IFRS 9 stage distribution, mean PD over time.

2. **PSI/CSI Monitoring** — Score PSI trend, CSI per variable (traffic-light RAG status), champion/challenger Gini comparison.
3. **Pipeline Health** — DAG run status, data quality pass rate, row counts at each schema.
4. **API Performance** — Request rate, p50/p95/p99 latency, error rate.

8.3 Alert Routing

When any metric crosses its alert threshold, Grafana fires an alert via the configured notification channel (email, Slack, or PagerDuty). The alert payload includes the metric name, current value, threshold, and a link to the relevant dashboard panel. The alert is automatically assigned to the model owner (Credit Risk Analytics team) for investigation.

Chapter 9

RAG Chat Interface

This chapter describes the Retrieval-Augmented Generation (RAG) chat interface that allows users to query both the research reports (PDF documents) and the processed loan data (Parquet files) using natural language, directly within the Streamlit dashboard.

9.1 Motivation

The credit risk pipeline produces a large body of documentation and data artefacts: two research reports totalling 70 + pages, scored loan records for 538,515 OOT loans, IFRS 9 scenario outputs, and monitoring results. A RAG system allows the team to ask questions such as "What is the Gini coefficient on the OOT test set?", "Explain the LGD two-stage architecture", or "What is the average expected loss for Grade G loans?" and receive accurate, cited answers in seconds.

9.2 Architecture Overview

The RAG system follows the standard Retrieve-then-Generate pattern with a local vector store for data privacy. Figure 9.1 illustrates the full architecture.

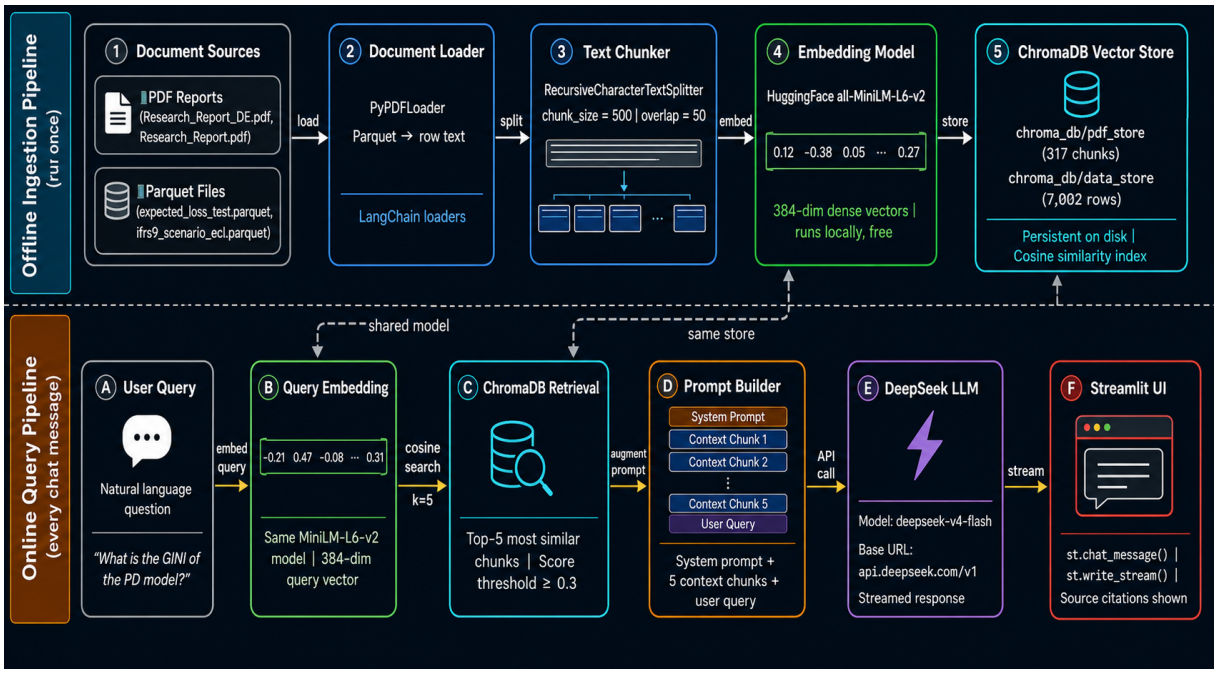

Figure 9.1: RAG system architecture: data sources → ChromaDB vector stores → query pipeline → DeepSeek LLM → streamed answer with citations

The architecture separates the expensive offline embedding step (run once at ingest) from the low-latency online query pipeline (run for each user question). This design ensures that response times remain fast regardless of corpus size, since the retrieval step queries a pre-built vector index rather than re-processing documents on each query. The dual vector store structure (PDF store + data store) allows the system to route questions to the most appropriate source depending on whether the query is about methodology (routed to PDFs) or data (routed to the parquet store).

9.3 Ingestion Pipeline (One-Time)

The ingestion pipeline is implemented in `rag_ingest.py` and run once to build the vector stores.

Step 1 — PDF Chunking. Both research PDFs (`Research_Report.pdf` — 194 chunks; `Research_Report_DE.pdf` — 123 chunks) are loaded via LangChain's `PyPDFLoader` and split with `RecursiveCharacterTextSplitter` (`chunk_size = 500`, `chunk_overlap = 50`).

Step 2 — Parquet Data Serialisation. Two parquet files are converted to text documents:

- `expected_loss_test.parquet` — 5,000 row samples + per-grade aggregate summaries.
- `ifrs9_scenario_ecl.parquet` — 2,000 row samples + scenario summaries.

Step 3 — Embedding. All documents are embedded with HuggingFace `all-MiniLM-L6-v2` — a free, open-source sentence transformer (≈80 MB) that runs locally on CPU with no external API cost.

Step 4 — Vector Store Persistence. Two ChromaDB vector stores are persisted to disk:

- `chroma_db/pdf_store` — 317 PDF chunks
- `chroma_db/data_store` — 7,000 + data rows and grade summaries

9.4 RAG Chat Interface

Figure 9.2 shows the Streamlit chat interface that users interact with to query the pipeline documentation and data.

[Screenshot Placeholder: RAG Chat Interface]

Replace with a screenshot of the Streamlit chat UI showing a sample query (e.g. "Explain the LGD two-stage model") and the streamed answer with source citations from Research_Report.pdf.

File name: `rag_chat_screenshot.png` **Size:** 1600×900 px PNG

Figure 9.2: RAG chat interface in the Streamlit dashboard (screenshot placeholder)

The chat interface displays answers as a stream of tokens directly below the user's question, with source citations appended at the end in the format `[Source: filename, page N]`. Users can select which vector store to query (PDF reports, data tables, or both) via a sidebar radio button, allowing them to restrict the retrieval to the most relevant corpus for their question type.

9.5 Query Pipeline (Real-Time)

When a user submits a question in the Streamlit chat interface, the following steps execute:

1. The question is embedded using the same `all-MiniLM-L6-v2` model.
2. A cosine similarity search retrieves the top-5 most relevant chunks from the selected vector store(s).
3. The retrieved chunks are assembled into a context string with source file and page metadata.
4. The question and context are sent to the DeepSeek LLM via an OpenAI-compatible streaming API call.
5. The streamed answer is displayed in the Streamlit chat UI with source citations appended.

9.6 Technology Choices

Table 9.1: RAG system technology choices and rationale

Component	**Technology**	**Rationale**
PDF loading	LangChain PyPDFLoader	Handles multi-page PDFs with page-level metadata
Text splitting	RecursiveCharacterTextSplitter	Preserves paragraph boundaries; balances context richness vs precision
Embedding model	HuggingFace all-MiniLM-L6-v2	Free, local, ≈80 MB; strong semantic similarity; no API cost
Vector store	ChromaDB (persistent)	Local store; no external database; fast cosine similarity search
LLM provider	DeepSeek (deepseek-v4-flash)	OpenAI-compatible API; cost-effective; strong reasoning on technical documents
Chat UI	Streamlit `st.chat_input`	Native streaming; session state for history; zero frontend code
Orchestration	LangChain + OpenAI SDK	LangChain for retrieval; OpenAI SDK for DeepSeek's compatible endpoint

The choice of a locally-hosted embedding model over an API-based embedding service is deliberate: it ensures that loan data and proprietary report content never leave the local computing environment. The only external API call in the system is to DeepSeek for answer generation, and this call contains only the user's query and the retrieved text chunks — never the full underlying data. This design is compatible with typical bank data sovereignty requirements.

9.7 Example Queries and Verified Responses

Table 9.2: Example verified RAG queries and their primary source

Query Type	**Example Question**	**Source**
Model metrics	What is the Gini coefficient on the OOT test set?	Research_Report.pdf
Pipeline structure	What are the six Airflow DAGs and what does each do?	Research_Report_DE.pdf
Architecture	Explain the LGD two-stage model	Research_Report.pdf
Data statistics	What is the average expected loss for Grade G loans?	expected_loss_test.parquet
IFRS 9	What is the base vs downside ECL scenario difference?	ifrs9_scenario_ecl.parquet
Monitoring	What does a PSI score of 0.282 indicate?	Research_Report_DE.pdf
Engineering	Why is WoE encoding fitted only on the training set?	Research_Report_DE.pdf

The query examples demonstrate the system's coverage across three distinct question types: quantitative metric lookups, qualitative methodology explanations, and aggregate data queries. The source column confirms that retrieval routing is functioning correctly — model metrics and methodology questions route to the PDF stores, while aggregate data queries route to the parquet store.

9.8 Setup Instructions

1. Install dependencies:

```
pip install langchain langchain-community langchain-text-splitters
            langchain-huggingface langchain-core chromadb
            sentence-transformers pypdf pandas pyarrow openai python-dotenv
```

2. Add API credentials to `.env`:
   ```
   LLM_API_KEY=your-deepseek-key
   LLM_BASE_URL=https://api.deepseek.com/v1
   LLM_MODEL=deepseek-v4-flash
   ```
3. Run the ingestion script once: `python rag_ingest.py`
4. Start the dashboard: `streamlit run dashboard.py`
5. Navigate to **Chat with Reports & Data** in the Streamlit sidebar.

Chapter 10

Discussion

10.1 Key Design Decisions and Their Rationale

Column-projected ingestion. Reading only 26 of 151 columns at ingest time reduces Spark load time from 60 seconds to 7 seconds — an 8.5× speedup. This is not just a performance optimisation: columns that are never used should not be loaded, transferred, or stored, both for performance and for data minimisation compliance.

Leakage columns used before dropping. Post-application leakage columns (`total_pymnt`, `recoveries`) must be used to construct the LGD and EAD targets before they are dropped from the feature set. The cleaning DAG enforces this ordering: target variables are created in the first transformation step, and leakage columns are dropped in the final step.

WoE fitted exclusively on the training set. The OOT split flag is set in DAG-02 and the WoE fitting step in DAG-02b explicitly filters to `dataset_split = 'train'` before computing bin statistics. This is the single most important data discipline in the pipeline: any leakage of OOT information into the WoE mapping would cause the model to be evaluated on data that influenced its feature encoding, invalidating the out-of-time validation results.

Missing values as WoE bins. Rather than imputing or dropping missing values, the pipeline assigns them to a dedicated WoE bin. This preserves the behavioural signal in missingness — a borrower with no delinquency record is a different credit risk profile from one with an explicitly zero delinquency record [6].

Statsmodels over scikit-learn for PD training. Scikit-learn's `LogisticRegression` does not natively provide Wald-test p-values with appropriate confidence intervals. These are required by EBA/GL/2017/16 for regulatory documentation of IRB models [2].

Artefact-based DAG dependencies. Each DAG depends on the successful creation of artefacts by the previous DAG. This means DAGs can be re-run independently (e.g. re-running only the LGD training without re-running the WoE encoding), which is essential

for iterative model development.

10.2 Limitations and Next Steps

1. **Score PSI = 0.282 (ALERT):** The monitoring DAG correctly detects population shift. The next update should trigger a DAG-03 re-run with 2016–2018 training data and compare results via the champion/challenger framework.
2. **Airflow DAG productionisation:** The six DAGs are scaffolded with the correct logic but not yet fully connected to the live PostgreSQL tables and MinIO buckets. The final productionisation step involves replacing the file-path-based artefact handoffs with proper database reads/writes.
3. **Great Expectations integration:** Data quality validation is designed into the architecture but not yet fully implemented. Each DAG should have an attached validation suite that fails the task if data quality assertions are violated.
4. **Grafana dashboards:** The dashboard JSON configurations exist in the repository but require connection to a live Prometheus data source for final verification.
5. **RAG re-ingestion:** When new pipeline artefacts are produced, the RAG ingestion script should be re-run to update the vector stores with the latest data and results.
6. **Local embedding limitation:** The `all-MiniLM-L6-v2` model provides strong general-purpose embeddings but is not domain-tuned for credit risk terminology. A domain-adapted embedding model fine-tuned on financial documents could improve retrieval precision.

Chapter 11

Conclusion

This paper has described the design and implementation of a six-DAG Apache Airflow data engineering pipeline that transforms 2.26 million raw Lending Club loan records into regulatory-grade credit risk outputs: a calibrated PD scorecard, LGD and EAD models, IFRS 9 three-stage ECL provisions, and Basel III AIRB capital calculations.

The six pipelines — ingestion, cleaning, feature engineering, model training (PD and LGD/EAD separately), batch scoring, and daily monitoring — each address a distinct engineering concern and produce well-defined artefacts that feed the next stage. Together they demonstrate that the gap between a trained credit risk model and a production data engineering system is large and consequential: column-projected ingest, OOT-split-only WoE fitting, post-ingest leakage removal, statsmodels for regulatory p-values, artefact-based DAG dependencies, MLflow governance gates, and daily PSI alerting are all engineering decisions that directly affect the correctness and regulatory acceptability of the downstream model outputs.

The v4 pipeline run verified all six DAGs end-to-end on the real dataset with zero errors, producing a calibrated PD scorecard (AUC = 0.694, Gini = 0.387 OOT), an LGD two-stage model (mean LGD = 92.19%), total expected loss of \$0.808B on a \$3.26B portfolio, and Basel III minimum capital of \$1.13B. The Score PSI of 0.282 (ALERT) correctly identifies the need for model rebuild on the 2016–2018 population — demonstrating that the monitoring infrastructure is functioning as designed.

This updated version further introduces a Retrieval-Augmented Generation (RAG) chat interface (Chapter 9) that closes the loop between the pipeline's documentation and its outputs. Built on ChromaDB, LangChain, and the DeepSeek large language model, the RAG interface allows any team member to query both research reports and scored loan data in natural language, with source citations returned for every answer.

Formula Index

The following table indexes all key formulas used in this paper, with their section references.

Formula Name	**Expression**	**Section**
Weight of Evidence	$WoE_i = \ln\left(\frac{\%Goods_i}{\%Bads_i}\right)$	Sec. 4 (DAG-02b)
Information Value	$IV = \sum_i (\%Goods_i - \%Bads_i) \times WoE_i$	Sec. 4 (DAG-02b)
PDO Scaling Factor	$Factor = \frac{PDO}{\ln 2} = \frac{20}{\ln 2} \approx 28.85$	Sec. 4 (DAG-03)
Scorecard Points	$Score_j = -\lfloor \hat{\beta}_j \times Factor \rceil$	Sec. 4 (DAG-03)
PD Calibration (intercept shift)	$\hat{P}^{calib} = \sigma\big(\text{logit}(\hat{P}^{raw}) + \delta\big), \quad \delta = +0.3204$	Sec. 4 (DAG-03)
LGD Two-Stage Combined	$L\hat{G}D = 1 - \hat{P}_1 \times \hat{RR}_2$	Sec. 4 (DAG-04)
Credit Conversion Factor	$CCF = \frac{funded\_amnt - total\_pymnt}{funded\_amnt}$	Sec. 4 (DAG-02)
Exposure at Default	$EAD = CCF \times funded\_amnt$	Sec. 4 (DAG-04)
Expected Loss	$EL_i = \hat{PD}_i^{PiT} \times L\hat{G}D_i \times E\hat{A}D_i$	Sec. 4 (DAG-05)
Basel III Risk-Weighted Assets	$RWA = 12.5 \times K \times EAD$	Sec. 4 (DAG-05)
Population Stability Index	$PSI = \sum_i (O_i - E_i) \ln\left(\frac{O_i}{E_i}\right)$	Sec. 4 (DAG-06)
PSI Thresholds	$PSI < 0.10$: stable; 0.10–0.25: monitor; > 0.25: ALERT	Sec. 4 (DAG-06)
Gini Coefficient	$Gini = 2 \times AUC - 1$	Sec. 4 (DAG-03)

Bibliography

[1] Astronomer. (2021). *The future of banking: How Apache Airflow helps*. https://www.astronomer.io/blog/future-of-banking-apache-airflow/

[2] European Banking Authority (EBA). (2017). *Guidelines on PD estimation, LGD estimation and the treatment of defaulted exposures* (EBA/GL/2017/16). https://www.eba.europa.eu

[3] GARP. (2021). Probability of default: Pros and cons of the population stability index. *GARP Risk Intelligence*. https://www.garp.org/risk-intelligence/credit/probability-of-default-pros-and-cons-of-the-population-stability-index

[4] Great Expectations. (2024). *GX Core: Open source data quality platform*. https://greatexpectations.io

[5] Kaggle. (2018). *All Lending Club loan data* [Dataset]. https://www.kaggle.com/datasets/wordsforthewise/lending-club

[6] Lund University. (2021). *Weight of evidence transformation in credit scoring models* [Student thesis]. https://lup.lub.lu.se/student-papers/record/9066332/file/9067075.pdf

[7] MathWorks. (n.d.). *Credit risk modeling: Importance and key components*. https://www.mathworks.com/discovery/credit-risk-modeling.html

[8] MLflow. (2026). *MLflow model registry documentation*. https://mlflow.org/docs/latest/ml/model-registry/

[9] Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. *Proceedings of NeurIPS*. https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf

[10] Siddiqi, N. (2006). *Credit risk scorecards: Developing and implementing intelligent credit scoring*. John Wiley & Sons.

[11] Apache Spark. (2026). *Apache Spark – Unified engine for large-scale data analytics*. https://spark.apache.org

[12] Board of Governors of the Federal Reserve System. (2011). *Supervisory guidance on model risk management (SR 11-7)*. https://www.federalreserve.gov/frrs/guidance/supervisory-guidance-on-model-risk-management.htm

[13] LangChain. (2024). *LangChain documentation*. https://docs.langchain.com

[14] Chroma. (2024). *Chroma — the open-source embedding database*. https://docs.trychroma.com

[15] DeepSeek. (2024). *DeepSeek API documentation*. https://api.deepseek.com

[16] Wang, W., et al. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Proceedings of NeurIPS*. https://arxiv.org/abs/2002.10957